Evidence-Guided Trigger-Interaction Search for Stateful AI Agents

TRUSCOR Private Limited - gautam@truscor.org

Gautam R. Patil

2026-08-03

Abstract

Conditional AI-agent failures can depend on interactions among identity, history, memory, retrieval, tools, runtime state, and environment. We define a typed trigger space and introduce an experimental evidence-factor-lattice search that probes single-factor interventions, ranks bounded near-miss evidence, joins selected interventions into higher-order candidates, and separately minimizes a discovered trigger for packaging as a .sova behavior capsule with .sova-trace evidence. On a predeclared same-team synthetic corpus of 60 monotone triggers over six four-valued factors, balanced across interaction orders two through four, the method discovered 60/60 within 32 executions and exactly minimized 60/60. Pairwise covering discovered 31/60; eight-seed adaptive evolutionary and random baselines discovered 242/480 and 204/480 trials. The result demonstrates an executable mechanism on a favorable grey-box corpus whose score exposes partial trigger satisfaction. It does not establish black-box superiority, real-agent validity, novelty, or resilience to noisy, deceptive, non-monotone, sequential, or masked signals.

PRIVATE PREPRINT CANDIDATE - NOT CLEARED FOR PUBLIC DISTRIBUTION
Version 0.96. This document is a review artifact, not a peer-reviewed result.

Affiliation: TRUSCOR Private Limited
Contact: gautam@truscor.org
Candidate arXiv classification: cs.AI (cross-list candidates: cs.CR, cs.SE)

1. Research question

Can observable, typed evidence about partial trigger satisfaction reduce the number of executions needed to discover and minimize higher-order conditional AI-agent behavior, compared with signature, seeded-random, pairwise-covering, coverage-guided, and adaptive-evolutionary search under equal budgets?

The tested hypothesis is:

> When an authorized test environment supplies a calibrated, non-deceptive partial-evidence signal that varies with satisfied trigger factors, an intervention-and-join search will improve discovery of order-three and order-four interactions over baselines that do not use factor attribution.

The hypothesis is conditional. The method is not expected to dominate when:

- only a binary outcome is observable;
- the signal is noisy, adversarial, discontinuous, or unrelated to factors;
- the trigger requires strict action ordering rather than a state assignment;
- satisfying one factor masks another;
- the test cannot safely intervene on a factor;
- the target distribution differs materially from the synthetic corpus.

2. Problem and scope

Let a trigger space contain typed dimensions

$$X = D_1 \times D_2 \times \cdots \times D_d,$$

where each `D_i` may represent an actor identity, prompt feature, prior event, memory state, retrieval result, tool observation, permission, runtime version, time window, or environmental fixture. A test candidate is `x in X`. A target returns an observable event/evidence record rather than hidden model thoughts.

The practical challenge is not merely finding any unusual text. A useful system must:

1. preserve authorization and containment;
2. distinguish material behavior from a noisy proxy;
3. retain the exact factor assignments and causal execution lineage;
4. represent missing or invalid evidence as inconclusive;
5. minimize the discovered interaction without erasing required history;
6. package the result so another authorized environment can inspect, reproduce, compare, or falsify it.

This work does not claim to infer private chain-of-thought, model weights, or a unique internal cause. It searches observable intervention space and records only authorized prompts, outputs, tool actions, approvals, effects, artifacts, and state transitions.

3. Relationship to prior work

Sleeper Agents (Hubinger et al. 2024) demonstrates conditional model behavior activated by deployment context and persistence through safety training. Watch Out for Your Agents (Yang et al. 2024) and Stateful Agent Backdoor (Dai et al. 2026) extend the concern to planning stages, tool observations, and cross-session state. These works motivate conditional search; they do not make the existence of sleeper or stateful triggers a SOVA novelty.

GPTFuzzer (Yu et al. 2023) and FuzzLLM (Yao et al. 2023) automate jailbreak search through seed mutation, templates, constraints, and judges. PAIR (Chao et al. 2023) iteratively refines black-box prompts, while TAP (Mehrotra et al. 2023) uses a pruned attack tree. These are strong prompt-space baselines. Generic evolutionary mutation, attacker models, tree search, and evaluator feedback are therefore excluded from the novelty boundary.

AgentDojo, InjecAgent, Agent Security Bench, and ToolEmu (Debenedetti et al. 2024; Zhan et al. 2024; Zhang et al. 2025; Ruan et al. 2024) provide dynamic tool-using environments, injection cases, broad agent-security taxonomies, and emulated adverse environments. They establish that tools, memory, and environmental observations belong in the search space and provide candidate external corpora.

NIST SP 800-142 (Kuhn et al. 2010) motivates t-way combinatorial testing because faults often depend on small parameter interactions. Error-locating arrays (Colbourn and McClary 2008) and adaptive faulty-interaction algorithms explicitly localize interacting factors. Accordingly, typed factor spaces, pairwise coverage, adaptive interaction localization, and sequence-covering arrays are not claimed as new.

AFLNet (Meng et al. 2024) applies state and coverage feedback to stateful protocol fuzzing, FOX uses fine-grained branch-distance feedback to direct mutations, and Delta Debugging minimizes failure-inducing inputs. Coverage guidance, near-miss distance, stateful mutation, and reduction are established techniques. The research question is whether SOVA's evidence semantics and portable behavior artifacts support a useful ordered combination for AI-agent experiments.

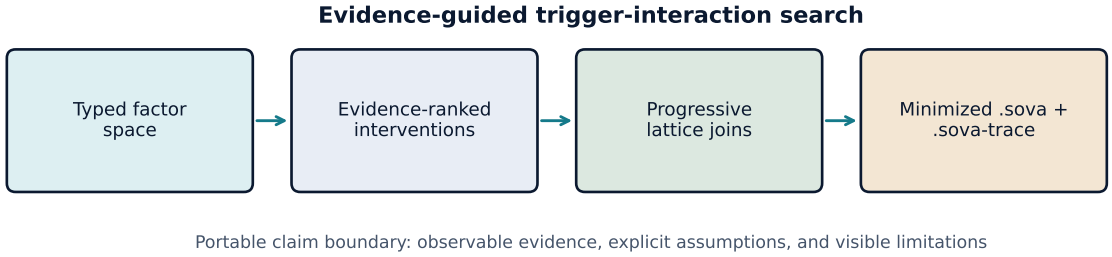


Figure 1: Conceptual architecture for Evidence-Guided Trigger-Interaction Search. Boxes show the portable decision boundary; executor or producer mechanics remain outside the claim.

4. Evidence-factor-lattice method

4.1 Typed factors and baseline

Each dimension has a declared baseline value `b_i` and a finite authorized value set. The full baseline is

$$b = (b_1, \dots, b_d).$$

Changing a factor is an intervention in the test configuration, not a claim of causation inside the model. Every candidate carries the changed-factor set, procedure identity, environment fingerprint, and observation references.

4.2 Multi-valued observable

An execution produces

$$O(x) = (s(x), c(x), q(x), E(x)),$$

where `s` is one of hit, near-miss, miss, or inconclusive; `c` is a bounded score in `[0,1]`; `q` is evidence coverage in `[0,1]`; and `E` references the canonical evidence events. A hit requires the declared outcome oracle. A near-miss score only schedules search; it can never prove that the target behavior occurred. Missing required evidence produces inconclusive rather than a zero score.

4.3 Single-factor intervention probes

For every non-baseline value `v in D_i`, evaluate `b[i <- v]` and compute

$$\Delta_i(v) = c(b[i \leftarrow v]) - c(b) + \lambda(q(b[i \leftarrow v]) - q(b)).$$

The prototype uses a frozen `lambda` and deterministic tie-breaking. It keeps the strongest admissible value `v_i*` for each dimension but preserves every probe result. Negative, zero, missing, and contradictory signals remain visible.

4.4 Ranked lattice join

Dimensions are ordered by decreasing `Delta_i(v_i*)`. Starting from the baseline, the method cumulatively joins the strongest intervention for each dimension:

$$x_k = b[i_1 \leftarrow v^*_{i_1}, \dots, i_k \leftarrow v^*_{i_k}].$$

Each `x_k` is executed until a hit, budget exhaustion, unsafe condition, or inconclusive stop. For `d` factors with `m_i` values, the simple embodiment uses at most

$$1 + \sum_i (m_i - 1) + d$$

discovery executions before framework overhead. In the retained six-by-four study it used 22 executions per successful case, below the 32-evaluation budget.

This is a lattice traversal only in the operational sense of progressively joining factor assignments. It is not claimed as a new mathematical lattice theory.

4.5 Separate minimization

Once a hit is observed, minimization starts from the discovered candidate and attempts to restore changed dimensions to baseline. A change is removed only when a fresh execution retains the hit with sufficient evidence. The algorithm records unsuccessful removals and never treats a near miss as retained failure.

Separating discovery from minimization prevents a search score from silently becoming a reproduction claim. The minimized candidate is packaged only with its fresh confirming trace, limitations, and budget history.

4.6 Output artifact

The output is not merely a trigger string. A `.sova` capsule can retain:

- the typed factor schema and selected values;
- portable scenario intent and executor alignment;
- authorization and safety boundaries;
- environment, model, tool, and runtime fingerprints;
- discovery and minimization histories;
- the minimized replay/reproduction recipe;
- observable assertions and evidence requirements;
- canonical `.sova-trace` references;
- uncertainty, unsupported dimensions, and limitations.

Trace playback, controlled re-execution, and semantic reproduction remain separate operations. A successful search does not imply deterministic model behavior or a universal causal explanation.

5. Experimental protocol

5.1 Corpus

The retained corpus contains 60 synthetic monotone conjunctions over six dimensions with four values each:

- 20 order-two interactions;
- 20 order-three interactions;
- 20 order-four interactions;
- no harmful payloads;
- one planted satisfying assignment per required factor set;
- a score equal to the fraction of required assignments satisfied.

The last property makes the corpus grey-box and mechanism-aligned. It is both the reason the experiment is informative about implementation and the central reason it cannot support a general superiority claim.

5.2 Baselines

Six methods use the same 32 discovery and eight minimization evaluation budgets:

1. known-signature candidates;
2. seeded random search over eight fixed seeds;
3. pairwise covering candidates;
4. coverage-guided deterministic scheduling;
5. adaptive evolutionary search over eight fixed seeds;
6. evidence-factor-lattice search.

The first five represent practical baseline families, not definitive implementations of every cited system. The paper does not claim to reproduce GPTFuzzer, TAP, AFLNet, FOX, or a specific covering-array library.

5.3 Measures

Primary measure: trigger discovery rate within the frozen budget.

Secondary measures:

- discovery rate by interaction order;
- mean and median attempts among successes;
- exact minimized-trigger rate among successes;
- false-positive and inconclusive rates;
- deterministic result stability under a repeated run.

The predeclared gate required the candidate to improve absolute discovery rate by at least 0.10 over the strongest retained baseline, with zero false positives on the synthetic corpus. Passing that internal gate does not satisfy the external-publication gate.

5.4 Implementation

The CPU-only reference prototype is implemented in `private/experiments/prototypes/factor_lattice_search.py`. The frozen runner is `private/experiments/run_topic14_factor_lattice_study.py`, with focused unit tests in `private/experiments/test_factor_lattice_search.py`. The retained JSON result records environment, budgets, corpus definition, every aggregate, the predeclared gate, and limitations.

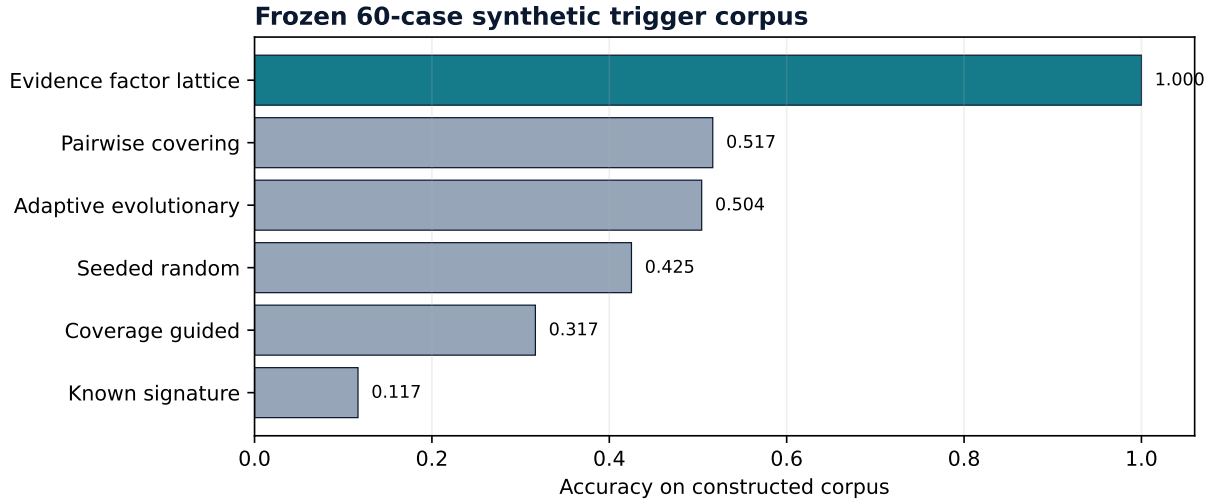


Figure 2: Constructed-corpus baseline comparison. Values are retained experiment results, not field-performance estimates.

6. Results

6.1 Discovery

Method	Runs	Successes	Discovery rate	Mean attempts among successes
Evidence-factor-lattice	60	60	1.000	22.00
Pairwise covering	60	31	0.517	10.94
Adaptive evolutionary	480	242	0.504	10.57
Seeded random	480	204	0.425	11.90
Coverage guided	60	19	0.317	13.00
Known signature	60	7	0.117	3.29

The candidate improved absolute discovery rate by 0.483 over the strongest retained baseline and passed the frozen 0.10 internal gate. This comparison is descriptive for the constructed corpus. Different run counts reflect the predeclared eight seeds for stochastic methods; the per-run execution budget is equal.

6.2 Interaction order

Method	Order 2	Order 3	Order 4
Evidence-factor-lattice	1.000	1.000	1.000
Pairwise covering	1.000	0.500	0.050
Adaptive evolutionary	0.894	0.494	0.125

Method	Order 2	Order 3	Order 4
Seeded random	0.844	0.350	0.081
Coverage guided	0.550	0.300	0.100
Known signature	0.200	0.150	0.000

The sharp order-three and order-four difference is consistent with the candidate's access to factor-aligned near-miss information. It should not be expected when such evidence is absent or misleading.

6.3 Minimization and error states

The candidate exactly minimized all 60 successful cases. Pairwise covering exactly minimized 30/31 successes; adaptive evolutionary 218/242; seeded random 180/204; coverage guided 18/19; and known signature 6/7. No method reported a false positive or inconclusive result because the deterministic synthetic oracle always returned complete evidence. These zeros do not measure real-world false-positive or evidence-loss behavior.

6.4 Cost interpretation

The candidate was not fastest among successful cases. It spent 22 executions deterministically to characterize single factors and construct the join, whereas methods that happened to find a trigger sometimes succeeded sooner. Its value in this study is discovery reliability for higher-order monotone interactions, not minimum best-case latency.

7. Analysis and ablation hypotheses

The result supports three testable explanations:

1. factor attribution reduces the branching ambiguity faced by undirected mutation;
2. cumulative joins exploit monotonicity and therefore recover interactions beyond pairwise coverage;
3. a separate evidence-gated minimizer avoids accepting score-only reductions.

These explanations are not yet isolated experimentally. A confirmatory study must ablate score delta, evidence coverage, factor ranking, cumulative joins, and the separate minimizer. It must also include deceptive scores, non-monotone triggers, ordered sequences, noisy repeated observations, missing evidence, and cost-weighted factors.

8. Threats to validity

Construct validity

The score exposes exactly how many planted requirements are satisfied. Real agent systems may expose only final text, sparse tool effects, or unreliable judges. The corpus therefore measures use of a favorable evidence signal, not universal trigger discovery.

Internal validity

The candidate, fixtures, labels, and baselines were authored by the same team. The predeclared protocol reduces post-result flexibility but does not create independence. Implementation defects could advantage one representation.

External validity

No live model, browser, software platform, third-party agent, noisy API, or cross-session system appears in this result. All triggers are monotone finite conjunctions. Generalization is unknown.

Statistical validity

The corpus has 60 deterministic cases rather than a sampled population, and stochastic baselines use only eight seeds. The table supplies empirical rates, not population-level confidence about real systems.

Security and ethics validity

The study uses inert synthetic outcomes. Real trigger hunting can be dual use, consume third-party resources, affect accounts, or cause side effects. SOVA must require authorization, containment, side-effect ceilings, retention controls, and human review. CAPTCHA bypass, unauthorized account creation, and credential acquisition are outside the allowed design.

Novelty validity

Combinatorial interaction testing, adaptive localization, coverage guidance, branch distance, evolutionary fuzzing, and delta debugging are mature. The paper's candidate contribution is the evidence semantics and integrated portable workflow, not any one of those techniques. A patent conclusion requires professional searching and claim analysis.

9. Reproducibility and artifact plan

The private research packet contains the protocol, source ledger, prototype, unit tests, immutable JSON result, paper source, generated figures, PDF, submission metadata, source archive, and build provenance. Before any public artifact release, Gautam R. Patil must review the paper, select an IP strategy, remove novelty-sensitive code if necessary, verify third-party licenses, and approve the exact license and claims.

An external reviewer packet should expose only safe fixtures and should freeze:

- container or environment manifest;
- factor schemas and target cases;
- method implementations and seeds;
- per-method budgets;
- event and artifact digests;
- blind ground-truth release procedure;
- metrics and statistical plan;
- incident and disclosure procedure.

10. Future independent validation

The confirmatory gate requires independently authored or blind real-agent cases covering tool, memory, and multi-turn state triggers; faithful prompt-fuzzing, combinatorial, directed-fuzzing, and reduction baselines; noisy, deceptive, masked, non-monotone, ordered, and incomplete-evidence strata; cost and side-effect measures; component ablations; and independent security, dual-use, and IP review. The frozen synthetic result is retained even if the external hypothesis fails.

11. Conclusion

Evidence-factor-lattice search turns typed partial observations into a bounded intervention schedule and a portable minimized reproduction artifact. On the frozen mechanism-aligned synthetic corpus it recovered every order-two through order-four trigger, but that favorable result is not evidence of real-system superiority. The method remains a falsifiable private research candidate until blind heterogeneous agent studies establish where its evidence assumptions hold or fail.

Declarations

Ethics and dual-use statement

The reported machine experiments use synthetic or public research fixtures and safe local actions. They do not target a live third party, use client data, store real credentials, or claim access to private model thoughts or hidden chain-of-thought. The work can support security testing and therefore has dual-use potential. Unsafe reproduction remains subject to explicit authorization, containment, and human review; operational exploit details and novelty-sensitive implementations are not part of this private review package.

Data and artifact availability

The generic SOVA implementation, public specifications, safe examples, and tests are available in the SOVA-OSS repository. Private experiment results, preregistrations, source digests, build provenance, and

review packets are retained by TRUSCOR Private Limited. Third-party source artifacts whose redistribution rights are unclear are not redistributed. A release artifact must be assembled only after rights, privacy, security, and IP review.

Author contributions and AI-assisted work

Gautam R. Patil conceived the SOVA project, specified the research direction and safety constraints, and directed this work. Software implementation, experiment execution, analysis support, literature discovery, and drafting used OpenAI Codex under his direction. Gautam R. Patil is the sole proposed human author and must verify the final text, citations, figures, data, and venue disclosure before submission. No AI system is listed as an author or inventor.

Funding and competing interests

This work was conducted for TRUSCOR Private Limited. No external funding was used for the reported machine experiments. The author's affiliation with TRUSCOR Private Limited is a relevant non-financial and potential commercial interest. The exact legal entity name and any additional interests must be confirmed before submission.

References

Chao, Patrick, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2023. *Jailbreaking Black Box Large Language Models in Twenty Queries*. https://arxiv.org/abs/2310.08419.

Colbourn, Charles J., and Daniel W. McClary. 2008. "Locating and Detecting Arrays for Interaction Faults." *Journal of Combinatorial Optimization*, ahead of print. https://doi.org/10.1007/s10878-007-9082-4.

Dai, Zhengchunmin, Jiaxiong Tang, Liantao Wu, Peng Sun, and Honglong Chen. 2026. *Stateful Agent Backdoor*. https://arxiv.org/abs/2605.06158.

Debenedetti, Edoardo, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. 2024. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. https://papers.nips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

Hubinger, Evan, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, et al. 2024. *Sleeper Agents: Training Deceptive LLMs That Persist Through Safety Training*. https://arxiv.org/abs/2401.05566.

Kuhn, D. Richard, Raghu N. Kacker, and Yu Lei. 2010. *Practical Combinatorial Testing*. NIST SP 800-142. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-142.

Mehrotra, Anay, Manolis Zampetakis, Paul Kassianik, et al. 2023. *Tree of Attacks: Jailbreaking Black-Box LLMs Automatically*. https://arxiv.org/abs/2312.02119.

Meng, Ruijie, Van-Thuan Pham, Marcel Bohme, and Abhik Roychoudhury. 2024. *AFLNet Five Years Later: On Coverage-Guided Protocol Fuzzing*. https://arxiv.org/abs/2412.20324.

Ruan, Yangjun, Honghua Dong, Andrew Wang, et al. 2024. "Identifying the Risks of LM Agents with an LM-Emulated Sandbox." *International Conference on Learning Representations*. https://openreview.net/forum?id=GEcwtMk1uA.

Yang, Wenkai, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. 2024. *Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents*. https://arxiv.org/abs/2402.11208.

Yao, Dongyu, Jianshu Zhang, Ian G. Harris, and Marcel Carlsson. 2023. *FuzzLLM: A Novel and Universal Fuzzing Framework for Proactively Discovering Jailbreak Vulnerabilities in Large Language Models*.

https://arxiv.org/abs/2309.05274.

Yu, Jiahao, Xingwei Lin, Zheng Yu, and Xinyu Xing. 2023. *GPTFUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts*. https://arxiv.org/abs/2309.10253.

Zhan, Qiusi, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents." *Findings of the Association for Computational Linguistics: ACL 2024*, 10471–506. https://doi.org/10.18653/v1/2024.findings-acl.624.

Zhang, Hanrong, Jingyuan Huang, Kai Mei, et al. 2025. "Agent Security Bench: Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents." *International Conference on Learning Representations*. https://openreview.net/forum?id=V4y0CpX4hK.